



LISTA DE EXERCÍCIOS - SISTEMAS INTELIGENTES APLICADOS

Lista de Exercícios 7

Redes Neurais (Perceptron)

DOCENTE: Thiago França Naves

DATA: 30/09/2026

ALUNO: Mayara Rodrigues Pereira e Leonardo José Reis Pinto

O treino das questões 1 e 2 foi feito no script `solucao.py` (Apêndice A), que também gera as tabelas passo a passo abaixo. Seguimos o modelo dos slides da aula: a entrada de bias é $x_0 = -1$ com peso w_0 , a soma é $net = w_0x_0 + w_1x_1 + w_2x_2$, a saída é $o = 1$ se $net > 0$ e $o = 0$ caso contrário, e a regra de aprendizado é

$$w_i \leftarrow w_i + \eta(t - o)x_i.$$

Os quatro exemplos são apresentados sempre na mesma ordem (00, 01, 10, 11), e o treino para quando uma época inteira passa sem erro.

Questão 1

Crie e faça o treinamento de uma rede neural do tipo perceptron que simule o funcionamento da porta OR.
a. Com os valores dos neurônios e pesos aleatórios. b. Considerando os seguintes pesos inicial $x_0 = -1$, $w_0 = 0$, $w_1 = 0$, $N = 0,2$. Quantas iterações são necessárias para a correta convergência dos pesos.

Resposta. A rede é um único neurônio com duas entradas e o bias. A porta OR dá 0 só para a entrada 00.

a) Sorteamos os pesos iniciais entre -1 e 1 (com semente fixa, para poder repetir): $w_0 = -0,35$, $w_1 = -0,70$ e $w_2 = 0,30$, com $\eta = 0,2$. Como w_1 começou negativo, a rede erra a entrada 10 por várias épocas seguidas, e cada erro sobe w_1 em $0,2$. O treino convergiu na época 7, depois de 10 correções, com $w_0 = 0,05$, $w_1 = 0,10$ e $w_2 = 0,30$.

Tabela 1. OR com pesos iniciais aleatórios.

Época	x_1	x_2	t	pesos antes			net	o	pesos depois		
				w_0	w_1	w_2			w_0	w_1	w_2
1	0	0	0	-0,35	-0,70	0,30	0,35	1	-0,15	-0,70	0,30
1	0	1	1	-0,15	-0,70	0,30	0,45	1	-0,15	-0,70	0,30
1	1	0	1	-0,15	-0,70	0,30	-0,55	0	-0,35	-0,50	0,30
1	1	1	1	-0,35	-0,50	0,30	0,15	1	-0,35	-0,50	0,30
2	0	0	0	-0,35	-0,50	0,30	0,35	1	-0,15	-0,50	0,30
2	0	1	1	-0,15	-0,50	0,30	0,45	1	-0,15	-0,50	0,30
2	1	0	1	-0,15	-0,50	0,30	-0,35	0	-0,35	-0,30	0,30
2	1	1	1	-0,35	-0,30	0,30	0,35	1	-0,35	-0,30	0,30
3	0	0	0	-0,35	-0,30	0,30	0,35	1	-0,15	-0,30	0,30
3	0	1	1	-0,15	-0,30	0,30	0,45	1	-0,15	-0,30	0,30
3	1	0	1	-0,15	-0,30	0,30	-0,15	0	-0,35	-0,10	0,30
3	1	1	1	-0,35	-0,10	0,30	0,55	1	-0,35	-0,10	0,30
4	0	0	0	-0,35	-0,10	0,30	0,35	1	-0,15	-0,10	0,30
4	0	1	1	-0,15	-0,10	0,30	0,45	1	-0,15	-0,10	0,30
4	1	0	1	-0,15	-0,10	0,30	0,05	1	-0,15	-0,10	0,30
4	1	1	1	-0,15	-0,10	0,30	0,35	1	-0,15	-0,10	0,30
5	0	0	0	-0,15	-0,10	0,30	0,15	1	0,05	-0,10	0,30
5	0	1	1	0,05	-0,10	0,30	0,25	1	0,05	-0,10	0,30
5	1	0	1	0,05	-0,10	0,30	-0,15	0	-0,15	0,10	0,30
5	1	1	1	-0,15	0,10	0,30	0,55	1	-0,15	0,10	0,30
6	0	0	0	-0,15	0,10	0,30	0,15	1	0,05	0,10	0,30
6	0	1	1	0,05	0,10	0,30	0,25	1	0,05	0,10	0,30
6	1	0	1	0,05	0,10	0,30	0,05	1	0,05	0,10	0,30
6	1	1	1	0,05	0,10	0,30	0,35	1	0,05	0,10	0,30
7	0	0	0	0,05	0,10	0,30	-0,05	0	0,05	0,10	0,30
7	0	1	1	0,05	0,10	0,30	0,25	1	0,05	0,10	0,30
7	1	0	1	0,05	0,10	0,30	0,05	1	0,05	0,10	0,30
7	1	1	1	0,05	0,10	0,30	0,35	1	0,05	0,10	0,30

b) Com $w_0 = w_1 = 0$ e também $w_2 = 0$ (a lista não cita w_2 , e usamos zero como os outros) e $\eta = 0,2$:

Tabela 2. OR com pesos iniciais zero e $\eta = 0,2$.

Época	x_1	x_2	t	pesos antes			net	o	pesos depois		
				w_0	w_1	w_2			w_0	w_1	w_2
1	0	0	0	0,00	0,00	0,00	0,00	0	0,00	0,00	0,00
1	0	1	1	0,00	0,00	0,00	0,00	0	-0,20	0,00	0,20
1	1	0	1	-0,20	0,00	0,20	0,20	1	-0,20	0,00	0,20
1	1	1	1	-0,20	0,00	0,20	0,40	1	-0,20	0,00	0,20
2	0	0	0	-0,20	0,00	0,20	0,20	1	0,00	0,00	0,20
2	0	1	1	0,00	0,00	0,20	0,20	1	0,00	0,00	0,20
2	1	0	1	0,00	0,00	0,20	0,00	0	-0,20	0,20	0,20
2	1	1	1	-0,20	0,20	0,20	0,60	1	-0,20	0,20	0,20
3	0	0	0	-0,20	0,20	0,20	0,20	1	0,00	0,20	0,20
3	0	1	1	0,00	0,20	0,20	0,20	1	0,00	0,20	0,20
3	1	0	1	0,00	0,20	0,20	0,20	1	0,00	0,20	0,20
3	1	1	1	0,00	0,20	0,20	0,40	1	0,00	0,20	0,20
4	0	0	0	0,00	0,20	0,20	0,00	0	0,00	0,20	0,20
4	0	1	1	0,00	0,20	0,20	0,20	1	0,00	0,20	0,20
4	1	0	1	0,00	0,20	0,20	0,20	1	0,00	0,20	0,20
4	1	1	1	0,00	0,20	0,20	0,40	1	0,00	0,20	0,20

Na primeira época a entrada 01 dá net = 0, saída 0, e o peso de x_2 sobe junto com o bias (w_0 vai para -0,2, porque $x_0 = -1$). Na segunda época a entrada 00 passa a dar net = 0,2 e é classificada errado, depois é a vez de 10. A última correção acontece no primeiro exemplo da terceira época, e a quarta época passa inteira sem erro, confirmando os pesos finais $w_0 = 0$, $w_1 = 0,2$ e $w_2 = 0,2$.

Então são necessárias **4 iterações de ajuste de pesos**, que acontecem ao longo de 3 épocas (9 apresentações de exemplos). Contando a época de verificação, o treino roda 4 épocas, ou 16 apresentações. Com esses pesos, a entrada 00 dá $\text{net} = 0$ e saída 0, e as outras três dão net de 0,2 ou 0,4 e saída 1, como na porta OR.

Questão 2

Crie e faça o treinamento de uma rede neural do tipo perceptron que simule o funcionamento da porta NAND.

Resposta. A NAND é o contrário da AND: dá 1 em tudo menos em 11. Usamos a mesma rede e os mesmos valores iniciais da questão 1b.

Tabela 3. NAND com pesos iniciais zero e $\eta = 0,2$.

Época	x_1	x_2	t	pesos antes			net	o	pesos depois		
				w_0	w_1	w_2			w_0	w_1	w_2
1	0	0	1	0,00	0,00	0,00	0,00	0	-0,20	0,00	0,00
1	0	1	1	-0,20	0,00	0,00	0,20	1	-0,20	0,00	0,00
1	1	0	1	-0,20	0,00	0,00	0,20	1	-0,20	0,00	0,00
1	1	1	0	-0,20	0,00	0,00	0,20	1	0,00	-0,20	-0,20
2	0	0	1	0,00	-0,20	-0,20	0,00	0	-0,20	-0,20	-0,20
2	0	1	1	-0,20	-0,20	-0,20	0,00	0	-0,40	-0,20	0,00
2	1	0	1	-0,40	-0,20	0,00	0,20	1	-0,40	-0,20	0,00
2	1	1	0	-0,40	-0,20	0,00	0,20	1	-0,20	-0,40	-0,20
3	0	0	1	-0,20	-0,40	-0,20	0,20	1	-0,20	-0,40	-0,20
3	0	1	1	-0,20	-0,40	-0,20	0,00	0	-0,40	-0,40	0,00
3	1	0	1	-0,40	-0,40	0,00	0,00	0	-0,60	-0,20	0,00
3	1	1	0	-0,60	-0,20	0,00	0,40	1	-0,40	-0,40	-0,20
4	0	0	1	-0,40	-0,40	-0,20	0,40	1	-0,40	-0,40	-0,20
4	0	1	1	-0,40	-0,40	-0,20	0,20	1	-0,40	-0,40	-0,20
4	1	0	1	-0,40	-0,40	-0,20	0,00	0	-0,60	-0,20	-0,20
4	1	1	0	-0,60	-0,20	-0,20	0,20	1	-0,40	-0,40	-0,40
5	0	0	1	-0,40	-0,40	-0,40	0,40	1	-0,40	-0,40	-0,40
5	0	1	1	-0,40	-0,40	-0,40	0,00	0	-0,60	-0,40	-0,20
5	1	0	1	-0,60	-0,40	-0,20	0,20	1	-0,60	-0,40	-0,20
5	1	1	0	-0,60	-0,40	-0,20	0,00	0	-0,60	-0,40	-0,20
6	0	0	1	-0,60	-0,40	-0,20	0,60	1	-0,60	-0,40	-0,20
6	0	1	1	-0,60	-0,40	-0,20	0,40	1	-0,60	-0,40	-0,20
6	1	0	1	-0,60	-0,40	-0,20	0,20	1	-0,60	-0,40	-0,20
6	1	1	0	-0,60	-0,40	-0,20	0,00	0	-0,60	-0,40	-0,20

O treino convergiu na época 6, depois de 11 correções, com $w_0 = -0,6$, $w_1 = -0,4$ e $w_2 = -0,2$. Conferindo com $\text{net} = 0,6 - 0,4x_1 - 0,2x_2$: a entrada 00 dá 0,6, a 01 dá 0,4 e a 10 dá 0,2, todas com saída 1, e a 11 dá exatamente 0, com saída 0. É a tabela da NAND.

Precisou de quase o triplo de correções do OR. Para a entrada 00 dar 1, o bias precisa empurrar a soma para cima, e com $x_0 = -1$ isso exige w_0 negativo. Ao mesmo tempo, w_1 e w_2 precisam ser negativos o bastante para a entrada 11 ficar em zero ou abaixo, e as correções de um exemplo desfazem em parte as do outro até os três pesos se acertarem.

No começo fizemos essas contas com números de ponto flutuante, e o treino da NAND parava na época 4. Conferindo à mão, a entrada 10 dava net igual a $0,4 - 0,4 = 0$, que deveria ter saída 0, mas o computador guardava um resto positivo minúsculo e respondia 1. Trocamos para frações exatas e o treino passou a seguir a regra à risca.

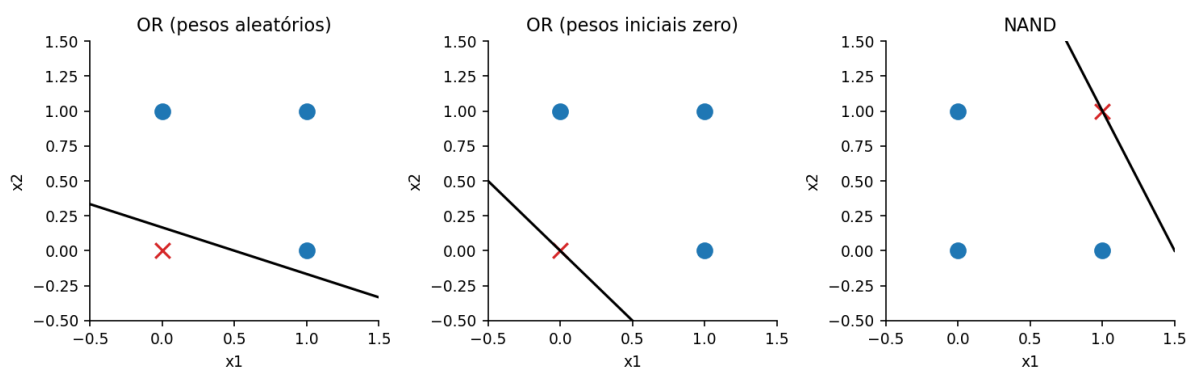


Figura 1. Retas de decisão $w_1x_1 + w_2x_2 = w_0$ encontradas. Círculos são saída 1 e X são saída 0.

Questão 3

Descreva as principais propriedades do modelo perceptron e porque ele só consegue modelar problemas linearmente separáveis com exemplos.

Resposta. O perceptron é um único neurônio: soma as entradas multiplicadas pelos pesos, soma o bias e passa o resultado por uma função degrau. Suas propriedades principais são:

- aprendizado supervisionado e incremental, com a regra $\Delta w_i = \eta (t - o) x_i$, que só mexe nos pesos quando a saída erra;
- saída binária, com uma decisão do tipo sim ou não;
- teorema de convergência: se as classes forem linearmente separáveis, o treino termina num número finito de correções, para qualquer $\eta > 0$;
- é rápido e simples, mas não dá uma medida de confiança e não tem camada escondida.

A decisão do perceptron é $\text{net} > 0$, ou seja, $w_1x_1 + \dots + w_nx_n > w_0$. A fronteira entre as duas respostas é $w_1x_1 + \dots + w_nx_n = w_0$, que é uma reta no plano (um hiperplano em mais dimensões). Tudo o que ele consegue fazer é deixar uma classe de cada lado de uma reta. O XOR é o exemplo clássico: os pontos 01 e 10 (saída 1) ficam numa diagonal e 00 e 11 (saída 0) na outra, e nenhuma reta separa as duas diagonais. No script, o treino do XOR erra os 4 exemplos em toda época a partir da terceira: cada correção desfaz a anterior e os pesos voltam ao mesmo ponto no fim de cada época. Para resolver o XOR é preciso combinar mais de um neurônio, com uma camada escondida, como na lista 8.

Questão 4

Porque as redes neurais são consideradas uma técnica de classificação supervisionada.

Resposta. Porque o treino precisa da resposta certa de cada exemplo. A regra de aprendizado compara a saída da rede o com a saída desejada t , e é o erro $t - o$ que decide quanto e em que direção os pesos mudam. Sem o rótulo não há erro para corrigir. É o mesmo esquema de um professor que corrige o aluno a cada resposta, daí o nome supervisionado. Existem redes não supervisionadas, como os mapas de Kohonen, mas o perceptron e o MLP com backpropagation dependem de exemplos rotulados.

Questão 5

O que se entende por aprendizado em Redes Neurais Artificiais?

Resposta. Aprender, numa rede neural, é ajustar os pesos das conexões (e os bias) a partir de exemplos, de forma que a rede passe a responder corretamente, inclusive para entradas que não viu no treino. O conhecimento da rede fica guardado nos pesos. O ajuste é iterativo: apresenta-se um exemplo, mede-se o erro e os pesos são corrigidos um pouco na direção que diminui esse erro, até a rede errar pouco o suficiente. A taxa de aprendizado η controla o tamanho de cada correção.

Questão 6

Para que servem as funções de ativação?

Resposta. A função de ativação decide a saída do neurônio a partir da soma ponderada. Ela limita a saída a uma faixa (0 a 1, -1 a 1) e, principalmente, introduz a não linearidade. Sem ela, cada camada seria uma combinação linear da anterior, e qualquer número de camadas se reduziria a uma única camada linear, que não resolve nem o XOR. Com funções não lineares, as camadas juntas conseguem aproximar fronteiras curvas. No backpropagation a função também precisa ter derivada, porque é ela que distribui o erro para as camadas de trás.

Questão 7

Quais critérios para definir o tipo de função de transferência utilizada (linear ou não linear). Discorra sobre algumas das funções de transferência utilizadas em redes neurais.

Resposta. A escolha depende de onde o neurônio está e do que a saída representa. Na saída da rede, a função segue o tipo de problema: linear para regressão (a saída pode ser qualquer número), sigmoide para duas classes (a saída vira uma probabilidade) e softmax para várias classes. Nas camadas escondidas a função tem que ser não linear, senão as camadas não acrescentam nada, e precisa ser derivável para o backpropagation. Também contam a faixa de valores das entradas e o custo de calcular.

- **Degrau:** 0 ou 1 conforme o sinal do net. É a do perceptron. Não tem derivada útil, por isso não serve para o backpropagation.
- **Linear:** $f(x) = x$. Usada na saída de regressões.
- **Sigmoide:** $f(x) = 1/(1 + e^{-x})$, saída entre 0 e 1, suave e derivável. Nas pontas a derivada fica quase zero, e em redes profundas o gradiente some (desvanecimento do gradiente).
- **Tangente hiperbólica:** saída entre -1 e 1, centrada no zero, o que costuma treinar mais rápido que a sigmoide, mas tem o mesmo problema nas pontas.
- **ReLU:** $f(x) = \max(0, x)$. Barata e sem saturação do lado positivo, é a mais usada hoje nas camadas escondidas. O problema é que neurônios com net sempre negativo param de aprender.
- **Softmax:** transforma as saídas de várias classes em probabilidades que somam 1.

Apêndice A. Código

```
pip install matplotlib
python solucao.py

# Lista 7 - Redes Neurais (Perceptron)
# Sistemas Inteligentes Aplicados - UTFPR
# Uso: python solucao.py

import os
import random
from fractions import Fraction as F
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

PASTA = os.path.dirname(os.path.abspath(__file__))
SAIDA = os.path.join(PASTA, "saidas")
os.makedirs(SAIDA, exist_ok=True)
plt.rcParams.update({"font.size": 9, "axes.spines.top": False,
                    "axes.spines.right": False})

ENTRADAS = [(0, 0), (0, 1), (1, 0), (1, 1)]
OR = [0, 1, 1, 1]
NAND = [1, 1, 1, 0]

def saida(w, x1, x2, x0=-1):
    # como nos slides: 1 se a soma ponderada for maior que zero
    net = w[0] * x0 + w[1] * x1 + w[2] * x2
    return (1 if net > 0 else 0), net

def treinar(alvos, w, eta, nome, max_epocas=100):
    # frações exatas: com float, 0,4 - 0,4 pode dar um resto positivo
    # e mudar a saída de um net que deveria ser zero
    w = [F(str(v)) for v in w]
    eta = F(str(eta))
    print(f" pesos iniciais w0={float(w[0]):.2f} w1={float(w[1]):.2f}"
          f" w2={float(w[2]):.2f}, "
          f" eta={eta}")
    passos = 0
    atualizacoes = 0
    linhas = []
    for epoca in range(1, max_epocas + 1):
        erros = 0
        for (x1, x2), t in zip(ENTRADAS, alvos):
            passos += 1
            o, net = saida(w, x1, x2)
            erro = t - o
            antes = list(w)
            if erro != 0:
                erros += 1
                atualizacoes += 1
                # regra do perceptron: w = w + eta (t - o) x
                w[0] += eta * erro * -1
                w[1] += eta * erro * x1
                w[2] += eta * erro * x2
            linhas.append((epoca, x1, x2, t, net, o, antes, list(w)))
        print(f" época {epoca}: {erros} erros, pesos"
              f" w0={float(w[0]):.2f} w1={float(w[1]):.2f}"
              f" w2={float(w[2]):.2f}")
        if erros == 0:
            break
    if erros == 0:
        print(f" {nome}: convergiu na época {epoca}, depois de {atualizacoes}"
              f" atualizações ({passos} apresentações de exemplos)")
    else:
        print(f" {nome}: não convergiu em {epoca} épocas")
    return w, linhas, epoca, atualizacoes

def num(v):
```

```

    return "$" + f"{float(v):.2f}".replace(".", "{,}") + "$"

def tabela(linhas, arquivo):
    # tabela em LaTeX com cada passo do treino, usada no relatório
    with open(os.path.join(SAIDA, arquivo), "w") as f:
        for ep, x1, x2, t, net, o, antes, depois in linhas:
            a = " & ".join(num(v) for v in antes)
            d = " & ".join(num(v) for v in depois)
            f.write(f"{ep} & {x1} & {x2} & {t} & {a} & {num(net)} & {o}"
                    f" & {d} \\\n")

def grafico(ax, w, alvos, titulo):
    for (x1, x2), t in zip(ENTRADAS, alvos):
        ax.scatter(x1, x2, s=80, marker="o" if t else "x",
                  color="tab:blue" if t else "tab:red")
    # reta w1 x1 + w2 x2 = w0
    xs = [-0.5, 1.5]
    if w[2] != 0:
        ax.plot(xs, [(w[0] - w[1] * x) / w[2] for x in xs], "k-")
    ax.set_xlim(-0.5, 1.5)
    ax.set_ylim(-0.5, 1.5)
    ax.set_xlabel("x1")
    ax.set_ylabel("x2")
    ax.set_title(titulo)

# questao 1a
print("[1a] OR com pesos aleatórios")
random.seed(7)
w_ini = [round(random.uniform(-1, 1), 2) for _ in range(3)]
w_or_a, linhas_a, _, _ = treinar(OR, w_ini, 0.2, "OR aleatório")
tabela(linhas_a, "tabela_or_aleatorio.tex")

# questao 1b
print("\n[1b] OR com x0=-1, w0=0, w1=0, w2=0 e eta=0,2")
w_or_b, linhas_b, ep_b, at_b = treinar(OR, [0, 0, 0], 0.2, "OR")
tabela(linhas_b, "tabela_or.tex")

# questao 2
print("\n[2] NAND com os mesmos pesos iniciais")
w_nand, linhas_n, _, _ = treinar(NAND, [0, 0, 0], 0.2, "NAND")
tabela(linhas_n, "tabela_nand.tex")

print("\n conferência:")
for nome, w, alvos in [("OR", w_or_b, OR), ("NAND", w_nand, NAND)]:
    res = [saida(w, x1, x2)[0] for x1, x2 in ENTRADAS]
    print(f" {nome}: saídas {res} esperado {alvos}")

# questao 3: XOR não converge
print("\n[3] XOR")
treinar([0, 1, 1, 0], [0, 0, 0], 0.2, "XOR", max_epocas=8)

fig, ax = plt.subplots(1, 3, figsize=(10, 3.2))
grafico(ax[0], w_or_a, OR, "OR (pesos aleatórios)")
grafico(ax[1], w_or_b, OR, "OR (pesos iniciais zero)")
grafico(ax[2], w_nand, NAND, "NAND")
fig.tight_layout()
fig.savefig(os.path.join(SAIDA, "retas.png"), dpi=150)
plt.close(fig)

```